# Numerical Methods Homework 5

B13902022 LAI, YU-CHI

**Octave is used in all implementations of this homework!**

## Problem 1

### (a)

By the equations in lecture materials and provided boudary conditions, we have the following linear systems:

$$\begin{bmatrix} 2h_0 & h_0 & 0 & \cdots & 0 \\ h_0 & 2(h_0+h_1) & h_1 & \cdots & 0 \\ 0 & h_1 & 2(h_1+h_2) & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & h_{n-1} & 2h_{n-1} \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix} = \begin{bmatrix} z_0 \\ z_1 \\ z_2 \\ \vdots \\ z_n \end{bmatrix}$$

Where the elements of vector $z$ are defined as:

$$\begin{aligned} z_0 &= \frac{3}{h_0}(a_1 - a_0) - 3f'(x_0) \\ z_j &= \frac{3}{h_j}(a_{j+1} - a_j) - \frac{3}{h_{j-1}}(a_j - a_{j-1}) \quad \text{for } j = 1, \ldots, n-1 \\ z_n &= 3f'(x_n) - \frac{3}{h_{n-1}}(a_n - a_{n-1}) \end{aligned}$$

After obtaining $c_j$'s, we can easily obtain $a, b, c$ using: $a_j = f(x_j)$, $b_j = \frac{1}{h_j}(a_{j+1} - a_j) - \frac{h_j}{3}(2c_j + c_{j+1})$, $d_j = \frac{c_{j+1} - c_j}{3h_j}$

---

```
% my_spline.m
function yq = my_spline(x, y, xq)
    x = x(:);
    n = length(x) - 1;

    fp_0 = y(1);
    fp_n = y(end);
    a = y(2:end-1);
    a = a(:);

    h = diff(x);
    A = zeros(n+1, n+1);
    z = zeros(n+1, 1);

    A(1, 1) = 2 * h(1);
    A(1, 2) = h(1);
    z(1) = (3 / h(1)) * (a(2) - a(1)) - 3 * fp_0;

    for i = 2:n
        A(i, i-1) = h(i-1);
```

```
        A(i, i)   = 2 * (h(i-1) + h(i));
        A(i, i+1) = h(i);

        term_right = (3 / h(i)) * (a(i+1) - a(i));
        term_left  = (3 / h(i-1)) * (a(i) - a(i-1));
        z(i) = term_right - term_left;
    end

    A(n+1, n)   = h(n);
    A(n+1, n+1) = 2 * h(n);
    z(n+1)      = 3 * fp_n - (3 / h(n)) * (a(n+1) - a(n));

    c = A \ z;

    b = zeros(n, 1);
    d = zeros(n, 1);
    for j = 1:n
        b(j) = (1 / h(j)) * (a(j+1) - a(j)) - (h(j) / 3) * (2 * c(j) + c(j+1));
        d(j) = (c(j+1) - c(j)) / (3 * h(j));
    end

    yq = zeros(size(xq));
    for k = 1:numel(xq)
        xq_val = xq(k);
        idx = sum(x <= xq_val);
        if idx == 0; idx = 1; elseif idx > n; idx = n; end
        dx = xq_val - x(idx);
        yq(k) = a(idx) + b(idx)*dx + c(idx)*dx^2 + d(idx)*dx^3;
    end
end
```

## (b)

I use the following script to test the correctness of `my_spline`. (Run `octave test_spline.m`, and the resulted image generated by the script can be seen below the code.)

```
% test_spline.m

f = @(x) sin(x);
df = @(x) cos(x);

x = linspace(0, 2*pi, 6);
xq = linspace(0, 2*pi, 200);

y_vals = f(x);
y_with_slopes = [df(x(1)), y_vals, df(x(end))];

yq_builtin = spline(x, y_with_slopes, xq);
yq_custom  = my_spline(x, y_with_slopes, xq);

max_err = max(abs(yq_builtin - yq_custom));
fprintf('Maximum difference between custom and built-in: %e\n', max_err);

fig = figure('visible', 'off');
```

```
19
20 set(fig, 'Position', [0, 0, 800, 600]);
21
22 plot(x, y_vals, 'ko', 'MarkerFaceColor', 'k', 'MarkerSize', 8, 'DisplayName', 'Data
   ↪  Points');
23 hold on;
24 plot(xq, f(xq), 'k-', 'LineWidth', 1, 'DisplayName', 'True Function \sin(x)');
25 plot(xq, yq_builtin, 'b-', 'LineWidth', 4, 'DisplayName', 'Built-in spline');
26 plot(xq, yq_custom, 'r--', 'LineWidth', 2, 'DisplayName', 'my\_spline');
27
28 legend('Location', 'northeastoutside');
29 title('Cubic Spline: Slope Boundary Conditions');
30 xlabel('x');
31 ylabel('f(x)');
32 grid on;
33 hold off;
34
35 output_filename = 'spline_plot.png';
36 print(fig, output_filename, '-dpng', '-r300');
37 fprintf('Plot successfully saved to %s\n', output_filename);
```

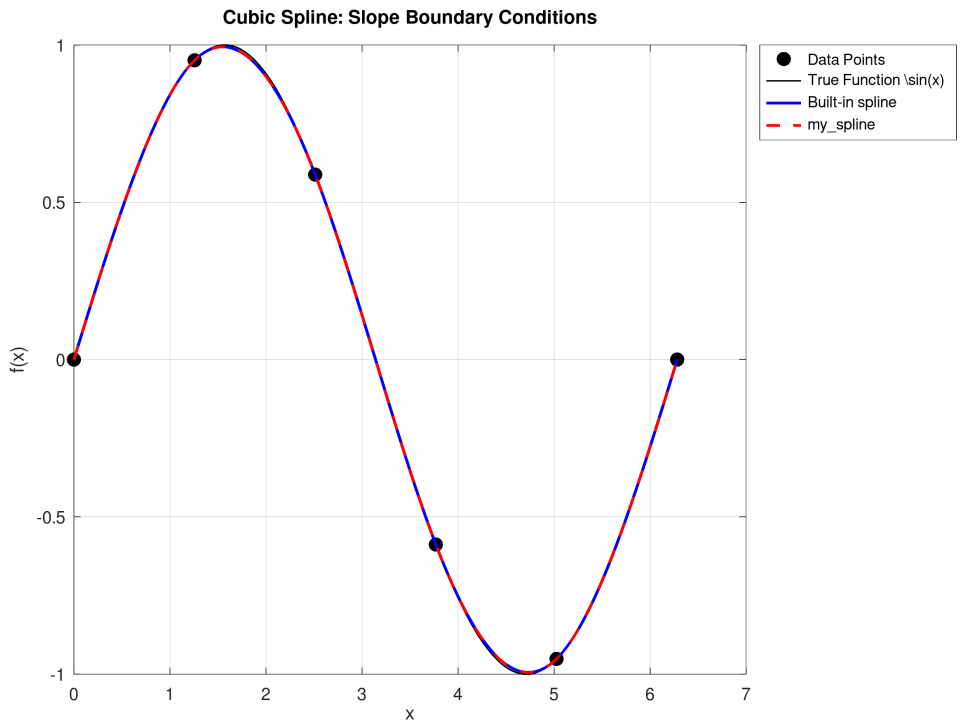


Figure 1: The output (Maximum difference between custom and built-in: 2.220446e-16)

## Problem 2

### (a)

$$E(a,b,c) = \sum_{i=1}^{m}(y_i - (ax_i^2 + bx_i + c))^2$$

$$\frac{\partial E}{\partial a} = 0 \Rightarrow a\sum_{i=1}^{m} x_i^4 + b\sum_{i=1}^{m} x_i^3 + c\sum_{i=1}^{m} x_i^2 = \sum_{i=1}^{m} x_i^2 y_i$$

$$\frac{\partial E}{\partial b} = 0 \Rightarrow a\sum_{i=1}^{m} x_i^3 + b\sum_{i=1}^{m} x_i^2 + c\sum_{i=1}^{m} x_i = \sum_{i=1}^{m} x_i y_i$$

$$\frac{\partial E}{\partial c} = 0 \Rightarrow a\sum_{i=1}^{m} x_i^2 + b\sum_{i=1}^{m} x_i + cm = \sum_{i=1}^{m} y_i$$

Therefore, we can obtain matrix $A$ and vector $z$ such that $A \begin{bmatrix} a \\ b \\ c \end{bmatrix} = z$ using the three equations above:

$$A = \begin{bmatrix} \sum_{i=1}^{m} x_i^4 & \sum_{i=1}^{m} x_i^3 & \sum_{i=1}^{m} x_i^2 \\ \sum_{i=1}^{m} x_i^3 & \sum_{i=1}^{m} x_i^2 & \sum_{i=1}^{m} x_i \\ \sum_{i=1}^{m} x_i^2 & \sum_{i=1}^{m} x_i & m \end{bmatrix}$$

$$z = \begin{bmatrix} \sum_{i=1}^{m} x_i^2 y_i \\ \sum_{i=1}^{m} x_i y_i \\ \sum_{i=1}^{m} y_i \end{bmatrix}$$

### (b)

```
% quad_least_square.m
function [a, b, c] = quad_least_square(x, y)
    x = x(:);
    y = y(:);

    m = length(x);

    sum_x  = sum(x);
    sum_x2 = sum(x.^2);
    sum_x3 = sum(x.^3);
    sum_x4 = sum(x.^4);

    sum_y   = sum(y);
    sum_xy  = sum(x .* y);
    sum_x2y = sum((x.^2) .* y);

    A = [sum_x4, sum_x3, sum_x2;
         sum_x3, sum_x2, sum_x;
         sum_x2, sum_x,  m];
    z = [sum_x2y;
         sum_xy;
         sum_y];

    coeffs = A \ z;

```

```
    a = coeffs(1);
    b = coeffs(2);
    c = coeffs(3);
end
```

---

## (c)

I use the following script to test the correctness of `quad_least_square`.

(Just run `octave test_quad_least_square.m`)

My function performs very well on $f_1(x) = e^x$, but performs poorly on $f_2 = \sin 2\pi x$ (See the resulted image generated by the script below the code.)

---

```
% test_least_square.m

% --- Function 1: Good Approximation ---
f1 = @(x) exp(x);
x1 = linspace(-1, 1, 20)'; % Generate 20 points
y1 = f1(x1) + 0.1 * randn(size(x1)); % Add a tiny bit of noise for realism

[a1, b1, c1] = quad_least_square(x1, y1);
y1_approx = a1*x1.^2 + b1*x1 + c1;

% --- Function 2: Poor Approximation ---
f2 = @(x) sin(2*pi*x);
x2 = linspace(0, 2, 50)'; % Generate 50 points
y2 = f2(x2) + 0.1 * randn(size(x2)); % Add a tiny bit of noise

[a2, b2, c2] = quad_least_square(x2, y2);
y2_approx = a2*x2.^2 + b2*x2 + c2;

figure('Position', [100, 100, 1000, 400]);

% Subplot 1: Good Fit
subplot(1, 2, 1);
plot(x1, y1, 'ko', 'MarkerFaceColor', 'k', 'DisplayName', 'Data Points (e^x + noise)');
hold on;
plot(x1, y1_approx, 'b-', 'LineWidth', 2, 'DisplayName', 'Quadratic Fit');
title('Good Approximation (Exponential)');
xlabel('x'); ylabel('y');
legend('Location', 'northwest');
grid on;

% Subplot 2: Poor Fit
subplot(1, 2, 2);
plot(x2, y2, 'ko', 'MarkerFaceColor', 'k', 'DisplayName', 'Data Points (\sin(2\pi x) +
↪  noise)');
hold on;
plot(x2, y2_approx, 'r-', 'LineWidth', 2, 'DisplayName', 'Quadratic Fit');
title('Poor Approximation (Sine Wave)');
xlabel('x'); ylabel('y');
legend('Location', 'northeast');
grid on;

print('least_squares_comparison.png', '-dpng', '-r300');
```

```
42 disp('Plot saved as least_squares_comparison.png');
```

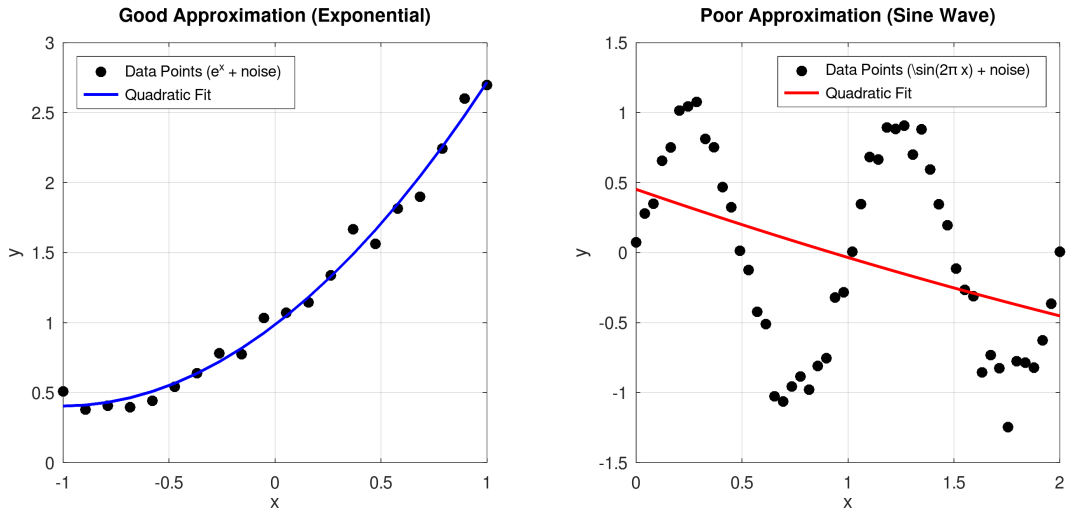


Figure 2: The performance on exponential function (Nice) and sine wave (Poor)